

# From Signals to Structure: How Memory Architecture Drives Language Emergence in LLM Agents

Yashar Talebirad<sup>1</sup> Eden Redman<sup>2</sup> Ali Parsaee<sup>1</sup> Osmar R. Zaiane<sup>1</sup>

<sup>1</sup> Alberta Machine Intelligence Institute, University of Alberta, Edmonton, Canada    <sup>2</sup> Network for Applied Technology, Edmonton, Canada  
talebira@ualberta.ca    eden@nat.ltd



ALIFE 2026

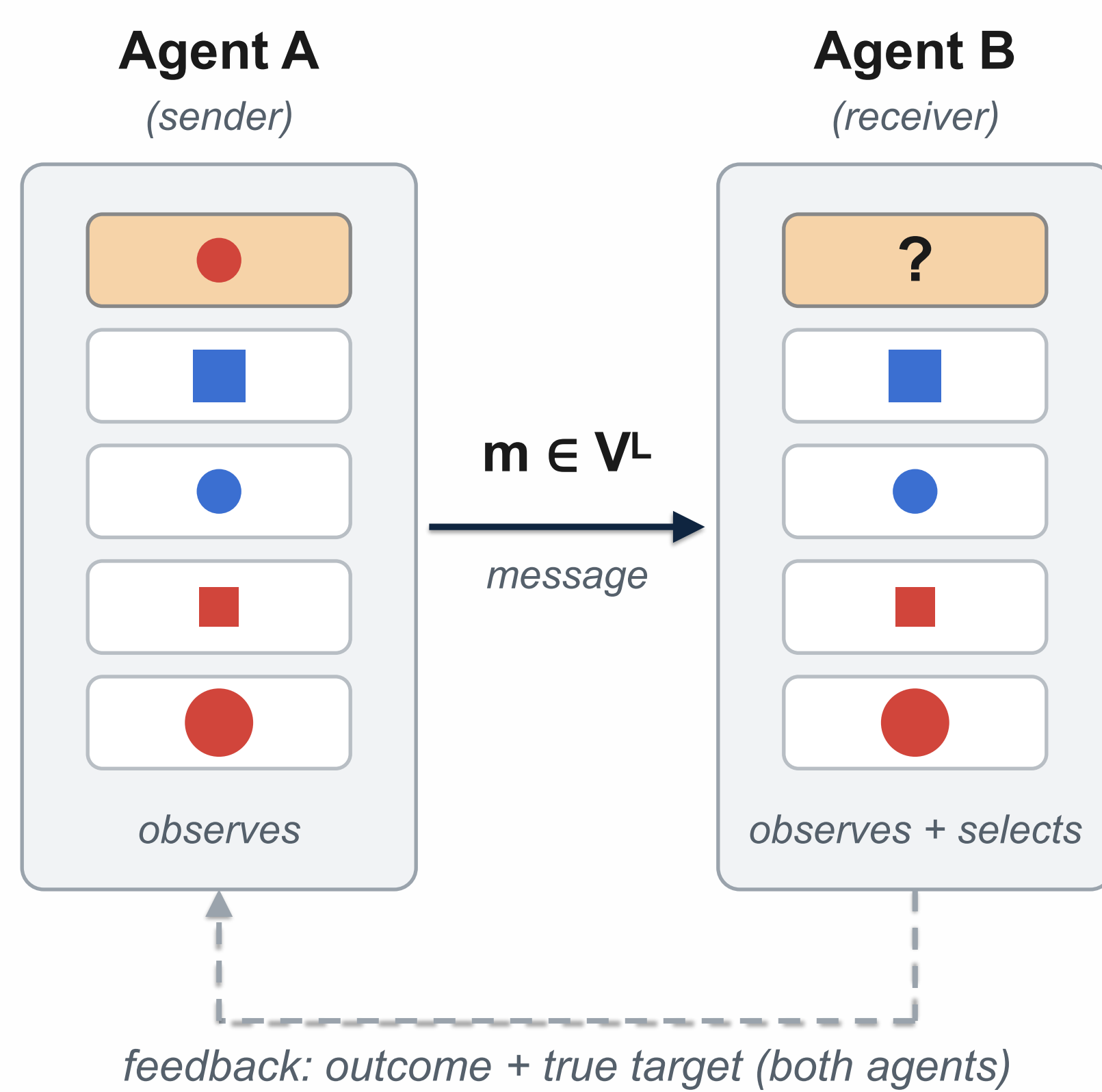


## Motivation

Two LLM agents must invent a shared language from scratch. In a Lewis signaling game the sender sees a target among four candidates and emits a fixed-length code; the receiver must recover the target from that code alone. No meanings are pre-assigned, so success means a language emerged.

Prior work treats the channel (how many distinct signals exist) as the variable that decides whether a code emerges. We ask instead what each agent can remember: hold the channel fixed, vary memory architecture, then sweep capacity from 4 to 125.

## The signaling game



$N = 200$  rounds. 8 objects = colour (red/blue)  $\times$  shape (circle/square)  $\times$  size (small/large). 4 candidates per round, so chance = 0.25. Sender emits  $L$  tokens from vocabulary  $V$ ; channel capacity =  $|V|^L$ , swept from 4 to 125. Base model for both agents: gpt-5.4-mini.

## Five memory architectures

| Condition     | Persistent | Private | Update    | Format                |
|---------------|------------|---------|-----------|-----------------------|
| memory_only   | No         | n/a     | n/a       | window only           |
| env_board     | Yes        | No      | env       | convention table      |
| scratchpad    | Yes        | Yes     | overwrite | free-form notebook    |
| codebook      | Yes        | Yes     | in-place  | slot list             |
| codebook_meta | Yes        | Yes     | in-place  | slot list + meta-note |

All five share the same base: a rolling window of the last 20 (message, target, success) interactions. They differ only in the persistent store layered on top. Only env\_board is shared; the rest are private to their owners.

The window is held fixed so differences reflect the persistent store, not the amount of raw history shown. Study 3 varies the window itself ( $m \in \{5, 10, 20, 40\}$ ) to confirm it is not driving the results.

## Results

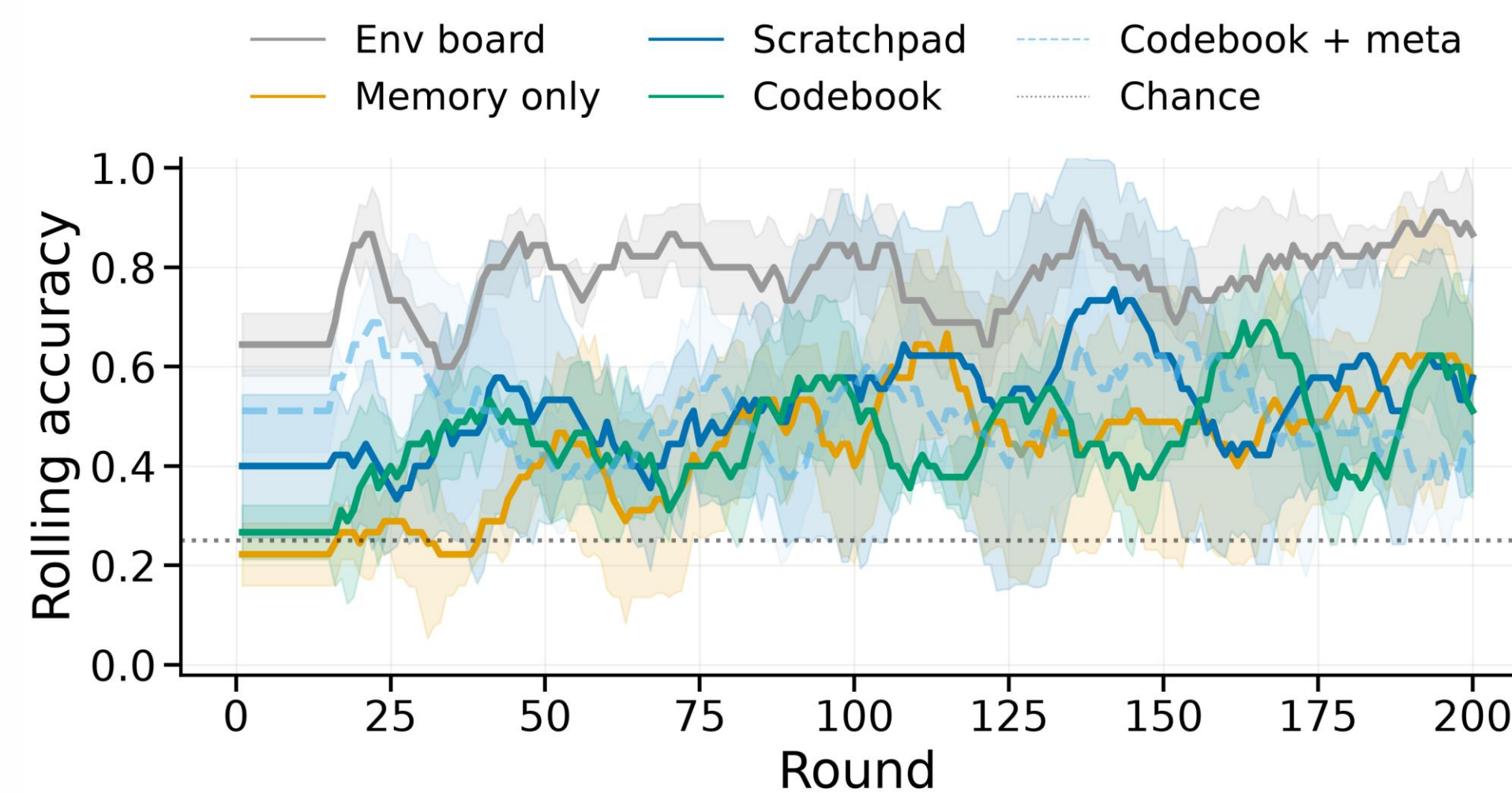


Fig 1 · Study 1: learning dynamics for all five architectures at a fixed channel (cap = 27, 3 seeds, 15-round rolling mean  $\pm$  std). Dotted line: chance (0.25).

An information-theoretic argument predicts an optimum at cap = 8, one signal per object. Memory architecture is the variable that argument leaves out.

TopSim = meaning-space vs signal-space distance correlation (compositionality). Best MI = token position vs object feature mutual information. Collision = objects sharing a modal message.

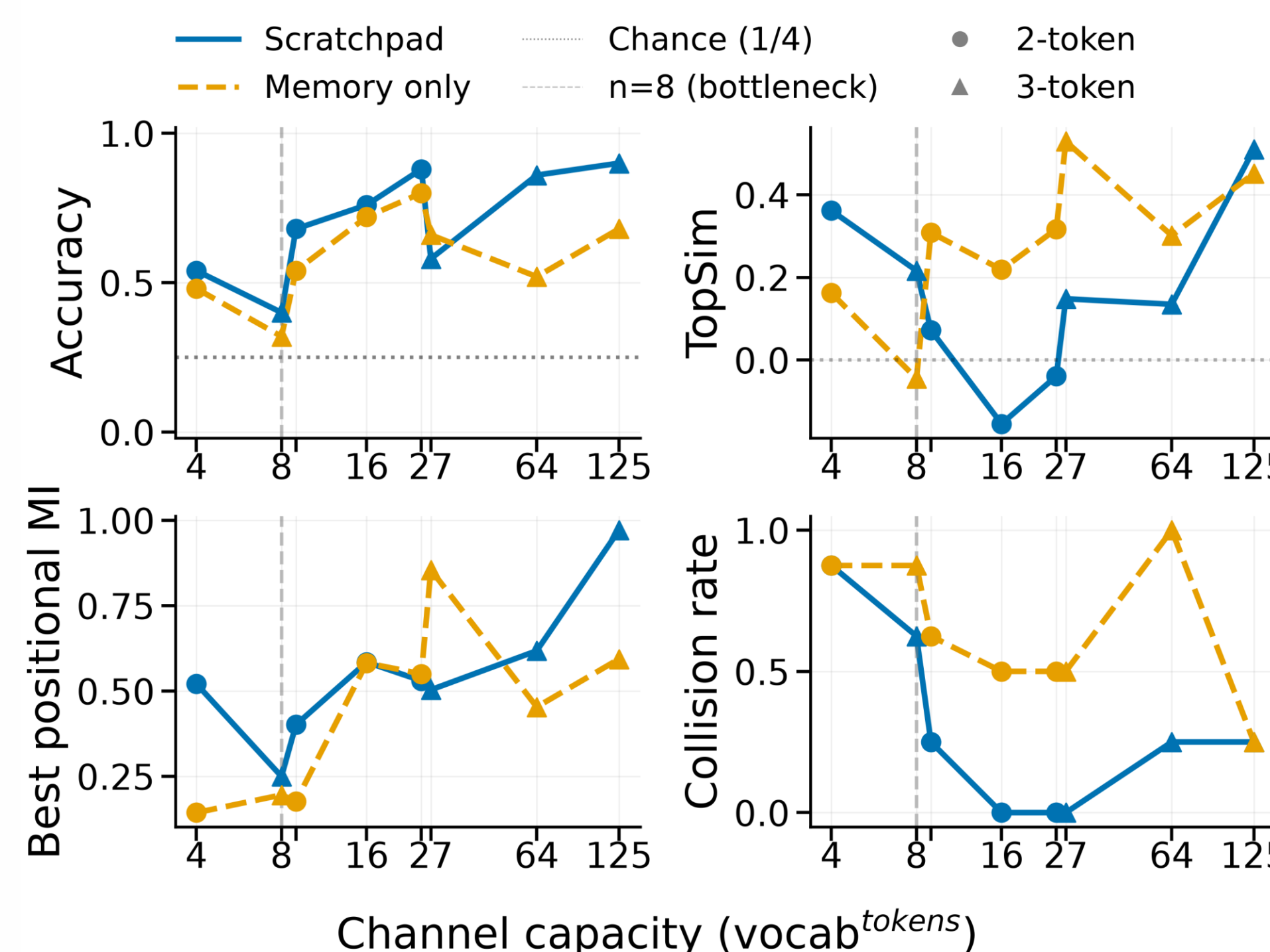


Fig 2 · Study 2: capacity curves for scratchpad and memory\_only (rounds 151–200, single seed). Dashed line at cap = 8: the predicted information-theoretic optimum for eight objects.

Opposite responses to surplus capacity: scratchpad climbs to 0.90 with zero collisions at cap 16–27; memory\_only degrades once its code space outgrows the window.

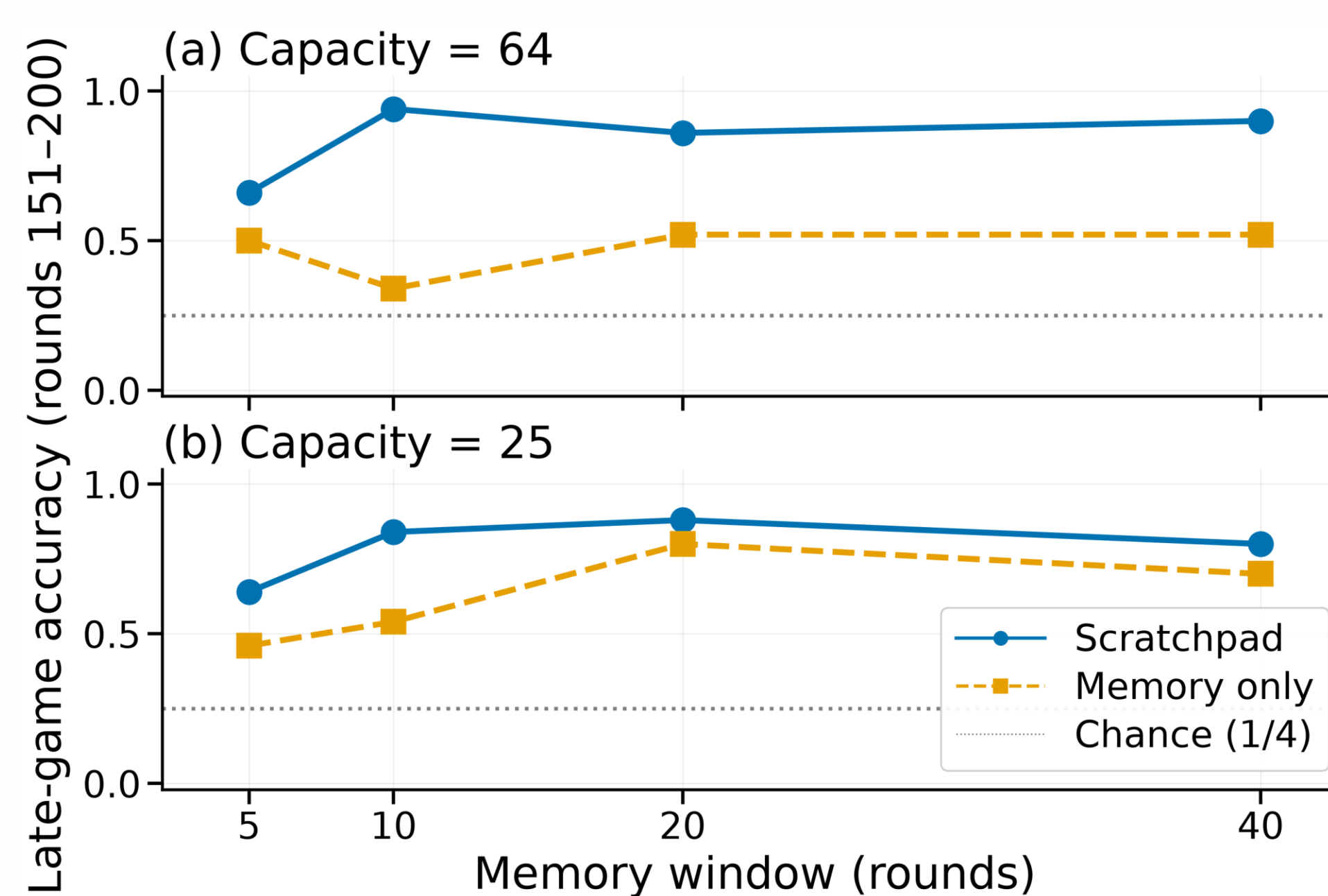


Fig 3 · Study 3: effect of memory-window size on late-game accuracy at cap = 64 (a) and cap = 25 (b), rounds 151–200, single seed.

At cap = 64 scratchpad reaches 0.94 on a 10-round window; memory\_only stays low at every size. Rolling-window memory fails only when the code space is large.

## Key findings

- Architecture beats capacity.** Hold the channel fixed at cap = 27 and the five memory architectures still span 0.46–0.83 late-game accuracy. Channel capacity alone cannot predict whether agents coordinate.
- Persistent private notebooks exploit surplus capacity.** Scratchpad rises 0.54  $\rightarrow$  0.90 as capacity grows and drives collisions to zero at cap = 16–27. Room beyond a perfect code lets agents repair early mistakes.
- Stateless agents (no persistent notebook) peak, then collapse.** memory\_only tops out at cap = 25 (0.80) and falls to 0.52 at cap = 64, where the collision rate hits 1.0: every object's modal message coincides with another's.
- cap = 8 is a fragility point, not an optimum.** Where signals exactly equal objects, outcomes go bimodal: runs either succeed ( $\geq 0.66$ ) or plateau ( $\leq 0.56$ ). Both architectures land on the same mean, 0.542: the only condition where the notebook advantage vanishes entirely.
- High accuracy is not language.** env\_board scores best on accuracy (0.827) with near-zero TopSim: it reads a shared lookup table instead of forming conventions. memory\_only has the best TopSim and MI but a 75% collision rate: locally structured, globally ambiguous.

## Conclusion

Channel capacity matters, but it does not decide outcomes. Memory architecture determines whether agents turn history into stable conventions. Language emerges only when capacity and memory support consolidation.

## Limitations

Single model (gpt-5.4-mini), fixed sender/receiver roles, 8 objects, 4-way discrimination. Most capacity cells run one seed; three key conditions replicate across seeds ( $n = 8$  at cap = 8).

READ THIS PAPER



Full paper (open access):  
[arXiv:2607.00233](https://arxiv.org/abs/2607.00233)